



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего
образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по учебной практике

УП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных
модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ Пицальникова Виктория Дмитриевна
(подпись) (фамилия, имя, отчество)

Группы ИПо-32

Руководитель практической подготовки от профильной организации

Преподаватель

(должность)

(подпись) Берестов Дмитрий Борисович
(фамилия, имя, отчество)

МП

«26» июня 2026 года

Руководитель практической подготовки от института

(подпись) Берестов Дмитрий Борисович
(фамилия, имя, отчество)

«27» июня 2026 года

Пермь, 2026 год

ОГЛАВЛЕНИЕ

ОГЛАВЛЕНИЕ	2
ВВЕДЕНИЕ	3
РАЗДЕЛ 1. РАЗРАБОТКА ТРЕБОВАНИЙ К ПРОГРАММНЫМ МОДУЛЯМ НА ОСНОВЕ АНАЛИЗА ПРОЕКТНОЙ И ТЕХНИЧЕСКОЙ ДОКУМЕНТАЦИИ НА ПРЕДМЕТ ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТ.....	5
1.1. Постановка задачи. Описание бизнес-процесса (AS IS).....	5
1.2. Описание процесса модернизации (TO BE)	5
1.3. Разработка технического задания.....	6
РАЗДЕЛ 2. ИНТЕГРАЦИЯ МОДУЛЕЙ В ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ..	8
2.1. Процесс разработки программного обеспечения.....	8
2.2. Диаграмма классов.....	9
2.3. Разработка программного модуля. Этапы разработки	11
РАЗДЕЛ 3. ОТЛАДКА ПРОГРАММНОГО МОДУЛЯ С ИСПОЛЬЗОВАНИЕМ СПЕЦИАЛИЗИРОВАННЫХ ПРОГРАММНЫХ СРЕДСТВ	16
3.1. Процесс отладки.....	16
РАЗДЕЛ 4. РАЗРАБОТКА ТЕСТОВЫХ НАБОРОВ И ТЕСТОВЫХ СЦЕНАРИЕВ ДЛЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	18
4.1. Процесс тестирования. Unit-тесты для классов-сервисов.....	18
4.2. Результаты тестирования.....	21
РАЗДЕЛ 5. ИНСПЕКТИРОВАНИЕ КОМПОНЕНТ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ НА ПРЕДМЕТ СООТВЕТСТВИЯ СТАНДАРТАМ КОДИРОВАНИЯ.....	23
5.1. Процесс инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.....	23
ЗАКЛЮЧЕНИЕ.....	25
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	26

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Учебная практика направлена на формирование у обучающихся первоначальных практических профессиональных умений в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем.

В настоящей работе представлен отчет о прохождении учебной практики студента-практиканта.

Основными видами деятельности являлись:

- разработка программного обеспечения
- выполнение отладки программного модуля с использованием специализированных программных средств
- осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения
- инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

Задачи практики:

- разрабатывать требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент
- выполнять интеграцию модулей в программное обеспечение
- выполнять отладку программного модуля с использованием специализированных программных средств

- осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения
- производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования
- выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам
- осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности
- использовать информационные технологии в профессиональной деятельности

РАЗДЕЛ 1. РАЗРАБОТКА ТРЕБОВАНИЙ К ПРОГРАММНЫМ МОДУЛЯМ НА ОСНОВЕ АНАЛИЗА ПРОЕКТНОЙ И ТЕХНИЧЕСКОЙ ДОКУМЕНТАЦИИ НА ПРЕДМЕТ ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТ

1.1. Постановка задачи. Описание бизнес-процесса (AS IS)

До разработки программного обеспечения процесс управления задачами выполнялся вручную: пользователь вёл записи задач в блокноте, таблицах Excel или в памяти. Это приводило к ряду проблем:

- невозможность быстрого поиска задач по ключевому слову или статусу
- отсутствие приоритизации и отслеживания срока выполнения
- сложность редактирования и удаления устаревших задач
- отсутствие возможности сохранить список задач в стандартном формате для переноса между устройствами

Таким образом, в состоянии «AS IS» пользователь не имел удобного инструмента для ведения структурированного списка задач с возможностью их классификации по статусу и приоритету.

1.2. Описание процесса модернизации (TO BE)

После разработки приложения TaskScheduler процесс управления задачами полностью автоматизирован. Система позволяет:

- добавлять задачи с указанием названия, описания, приоритета, срока выполнения и статуса
- просматривать полный список задач в табличном виде

- фильтровать задачи по статусу (Новая, В процессе, Завершена) и искать по тексту названия или описания
- редактировать существующие задачи через отдельное диалоговое окно
- удалять задачи из списка
- сохранять и загружать список задач из файлов формата JSON и XML

Сравнительный анализ процессов «AS IS» и «TO BE» показывает, что применение приложения TaskScheduler устраняет все выявленные проблемы ручного учёта и обеспечивает структурированное, быстрое и надёжное управление задачами.

1.3. Разработка технического задания

Техническое задание разработано в соответствии с требованиями ГОСТ 19.201-78 и ГОСТ 34.602-89.

Цель разработки: создать настольное приложение для управления задачами, реализованное в виде WPF-приложения с отдельной библиотекой классов, содержащей основную бизнес-логику.

Основные требования к программному продукту:

- добавление задачи с атрибутами: название (обязательное), описание, приоритет (Low / Normal / High), срок выполнения (дата, обязательно в будущем), статус (Created / InWork / Completed)
- просмотр списка всех задач в виде таблицы DataGridView
- фильтрация задач по статусу через выпадающий список
- поиск задач по тексту в названии или описании
- редактирование выбранной задачи
- удаление выбранной задачи

- сохранение и загрузка списка задач в/из файла формата JSON и XML
- основная логика приложения размещена в динамической библиотеке классов TaskSchedulerData
- для классов библиотеки написаны модульные тесты на платформе MSTest

Инструменты разработки: IDE Visual Studio 2022, платформа .NET 8.0, язык программирования C#, графическая подсистема WPF, платформа MSTest.

РАЗДЕЛ 2. ИНТЕГРАЦИЯ МОДУЛЕЙ В ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

2.1. Процесс разработки программного обеспечения

Для разработки приложения была выбрана итерационная модель разработки программного обеспечения. Её выбор обусловлен следующими причинами:

- небольшой объём разрабатываемого приложения позволяет выделить несколько чётких итераций: проектирование модели данных, реализация библиотеки классов, создание WPF-интерфейса, написание тестов
- после каждой итерации можно проверить работоспособность реализованного функционала и скорректировать следующие шаги
- итерационная модель гибче каскадной: при обнаружении проблем в одной части не требуется переработки всего проекта

Используемые инструменты:

- Visual Studio 2022
- платформа: .NET 8.0
- язык программирования: C#
- графическая подсистема: WPF (Windows Presentation Foundation)
- платформа тестирования: MSTest
- системы контроля версий: Git / GitHub

Среда программирования Visual Studio 2022 обеспечивает встроенную поддержку отладчика, средства рефакторинга, интеграцию с Git, а также визуальный дизайнер XAML для разработки пользовательского интерфейса WPF.

Структура решения состоит из трёх проектов:

- TaskScheduler - WPF-приложение (пользовательский интерфейс)
- TaskSchedulerData - библиотека классов (модель данных, сервисы, интерфейсы)
- TaskSchedulerDataTests - проект модульных тестов MSTest

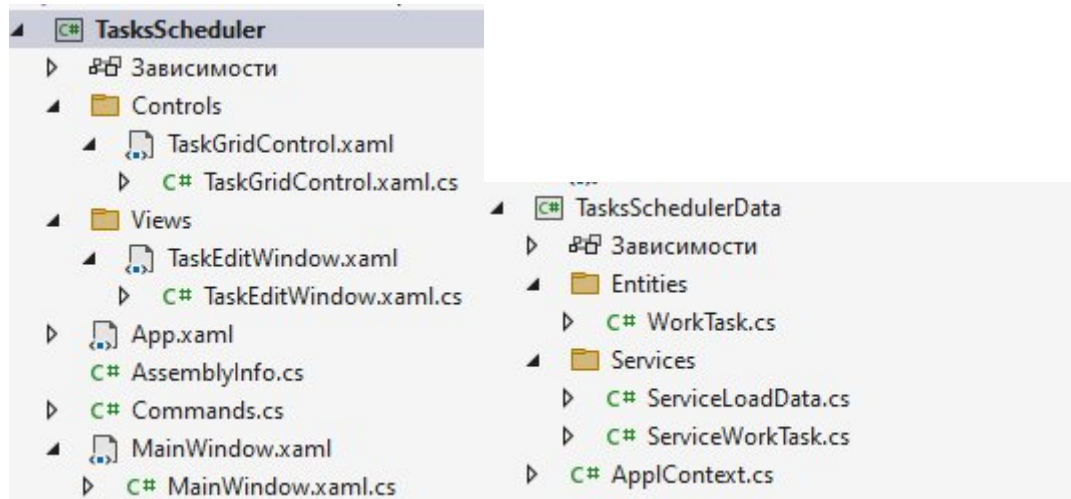


Рисунок 1 - структура проекта.

2.2. Диаграмма классов

Диаграмма классов отражает архитектуру библиотеки TaskSchedulerData и её взаимодействие с WPF-модулем.

Основные классы и интерфейсы:

- WorkTask - модель данных задачи. Содержит свойства: TaskId (Guid), TaskName (string), TaskDescription (string), Priority (PriorityLevel), DeadlineDate (DateTime), State (TaskState). Перечисления PriorityLevel (Low, Normal, High) и TaskState (Created, InWork, Completed) определены в том же файле.
- ApplContext - контейнер данных. Хранит коллекцию List<WorkTask> и заполняет её тестовыми данными при инициализации.

- `IServiceWorkTask` - интерфейс сервиса управления задачами. Объявляет методы: `Add`, `Update`, `Delete`, `GetTasks`, `GetStateTasks`, `GetTextTasks`.
- `ServiceWorkTask` - реализация `IServiceWorkTask`. Выполняет валидацию и операции над коллекцией через `AppContext`.
- `IServiceLoadData` - интерфейс сервиса файловых операций. Объявляет методы `Save` и `Load`.
- `ServiceLoadDataJson` - реализация `IServiceLoadData` для формата JSON (`System.Text.Json`).
- `ServiceLoadDataXml` - реализация `IServiceLoadData` для формата XML (`XmlSerializer`).
- `TaskGridControl` - пользовательский элемент управления WPF. Использует `ServiceWorkTask`, `ServiceLoadDataJson` и `ServiceLoadDataXml`.
- `TaskEditWindow` - диалоговое окно ввода/редактирования задачи.
- `Commands` - статический класс с маршрутизируемыми командами WPF (`RoutedCommand`).

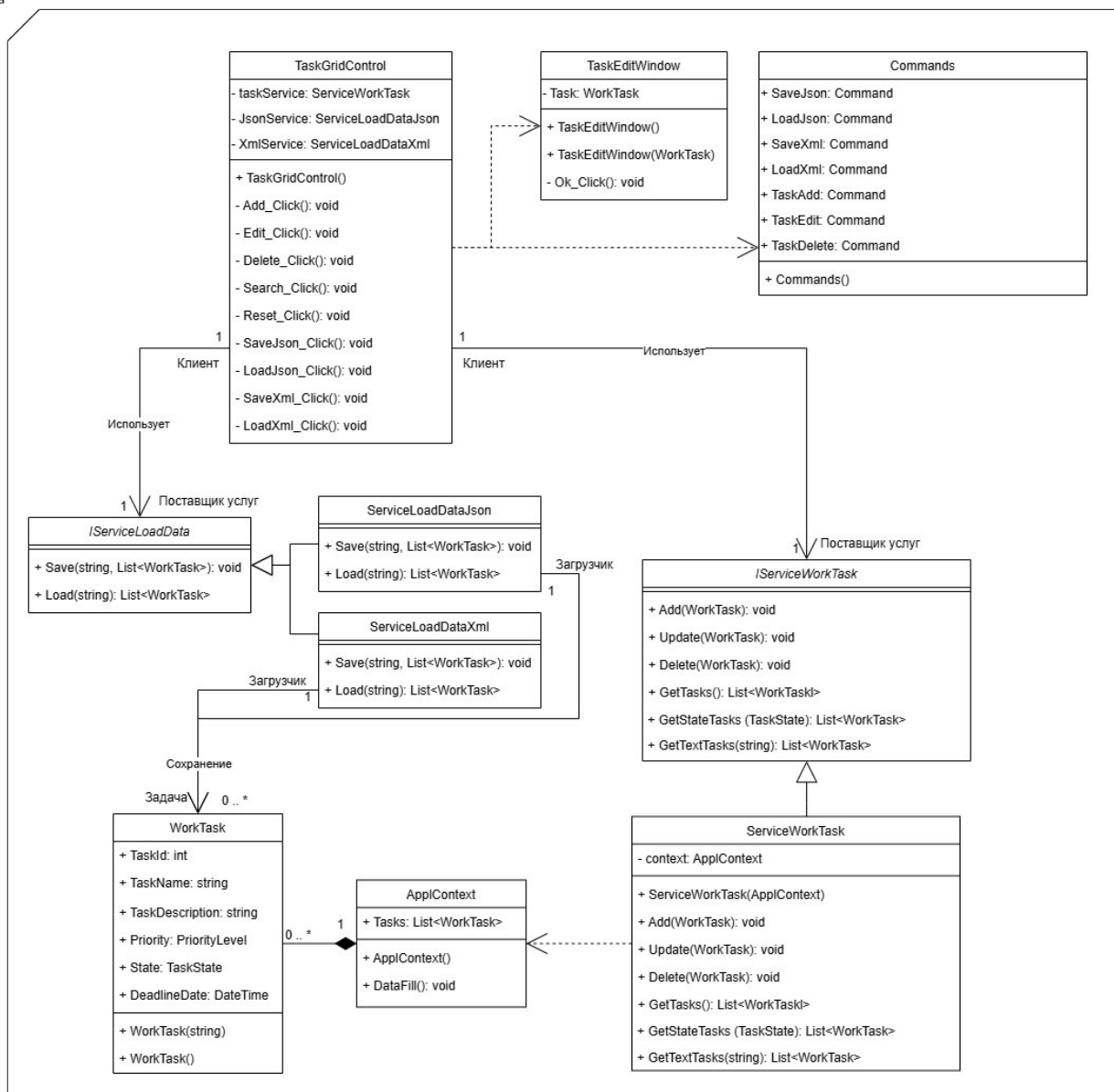


Рисунок 2 - диаграмма классов.

2.3. Разработка программного модуля. Этапы разработки

Разработка выполнялась поэтапно:

Этап 1. Создание проекта. В Visual Studio создан проект WPF-приложения TaskScheduler на платформе .NET 8.0. Затем к решению добавлена библиотека классов TaskSchedulerData, и в основной проект добавлена ссылка на неё.

Этап 2. Создание модели данных. В библиотеке TaskSchedulerData в папке Entities создан класс WorkTask с перечислениями PriorityLevel и TaskState, а также класс ApplContext, хранящий коллекцию задач.

Класс WorkTask:

```
public class WorkTask
{
    public Guid TaskId { get; set; } = Guid.NewGuid();
    public string TaskName { get; set; }
    public string TaskDescription { get; set; }
    public PriorityLevel Priority { get; set; }
    public DateTime DeadlineDate { get; set; }
    public TaskState State { get; set; } = TaskState.Created;
}
```

Этап 3. Создание сервиса управления задачами. Написан интерфейс IServiceWorkTask и его реализация ServiceWorkTask. Сервис выполняет валидацию задачи (проверку на null, пустое имя, корректность даты) и реализует методы CRUD, фильтрации по статусу и поиска по тексту.

Метод Add с валидацией:

```
public void Add(WorkTask task)
{
    ValidateTask(task);
    _context.Tasks.Add(task);
}
```

```
private static void ValidateTask(WorkTask task)
{
    if (task == null) throw new ArgumentNullException(nameof(task));
    if (string.IsNullOrEmpty(task.TaskName))
        throw new ArgumentException("Имя задачи не задано.");
    if (task.DeadlineDate <= DateTime.Now)
```

```

        throw new ArgumentException("Дата должна быть больше
текущей.");
    }

```

Этап 4. Создание сервисов файловых операций. Написан интерфейс IServiceLoadData и два его класса-реализации: ServiceLoadDataJson (использует System.Text.Json) и ServiceLoadDataXml (использует XmlSerializer). Каждый класс реализует методы Save и Load с проверкой существования файла.

Этап 5. Создание WPF-приложения. В проекте TaskScheduler созданы следующие компоненты:

- Commands.cs - статический класс с командами RoutedCommand для операций с файлами (SaveJson, LoadJson, SaveXml, LoadXml) и задачами (TaskAdd, TaskEdit, TaskDelete);
- Views/TaskEditWindow.xaml - диалоговое окно для добавления и редактирования задачи, содержащее поля: название (TextBox), описание (TextBox), приоритет (ComboBox), срок (DatePicker), статус (ComboBox) и кнопку ОК;
- Controls/TaskGridControl.xaml - пользовательский элемент управления с DataGrid для отображения задач, полем поиска, выпадающим списком статусов и кнопками «Добавить», «Изменить», «Удалить»;
- MainWindow.xaml - главное окно с меню (Файл → операции с JSON/XML; Задача → CRUD-команды) и встроенным элементом TaskGridControl.

Фрагмент разметки главного окна:

```

<DockPanel>
    <Menu DockPanel.Dock="Top">
        <MenuItem Header="Файл">
            <MenuItem Header="Сохранить JSON"
Command="comm:Commands.SaveJson"/>
            <MenuItem Header="Загрузить JSON"
Command="comm:Commands.LoadJson"/>

```

```

        <MenuItem                Header="Сохранить"                XML"
Command="comm:Commands.SaveXml"/>

        <MenuItem                Header="Загрузить"                XML"
Command="comm:Commands.LoadXml"/>

    </MenuItem>

</Menu>

<uc:TaskGridControl/>

</DockPanel>

```

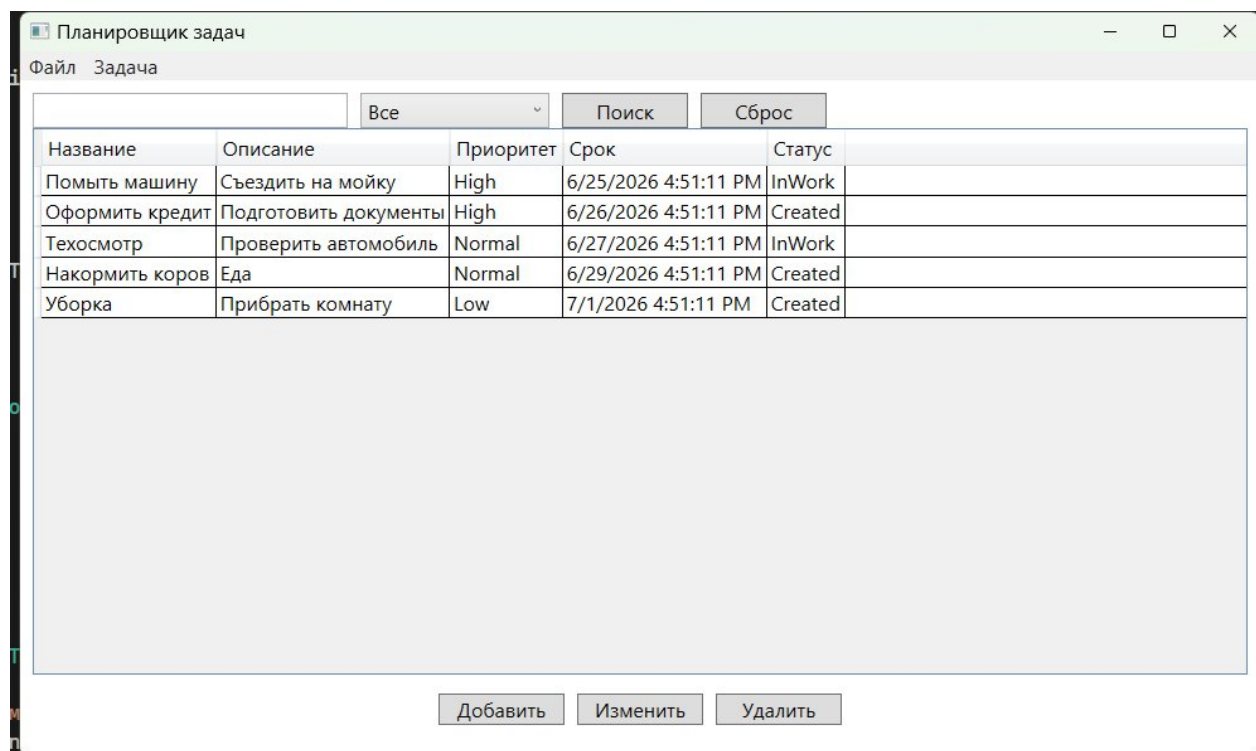


Рисунок 3 - главное окно приложения.

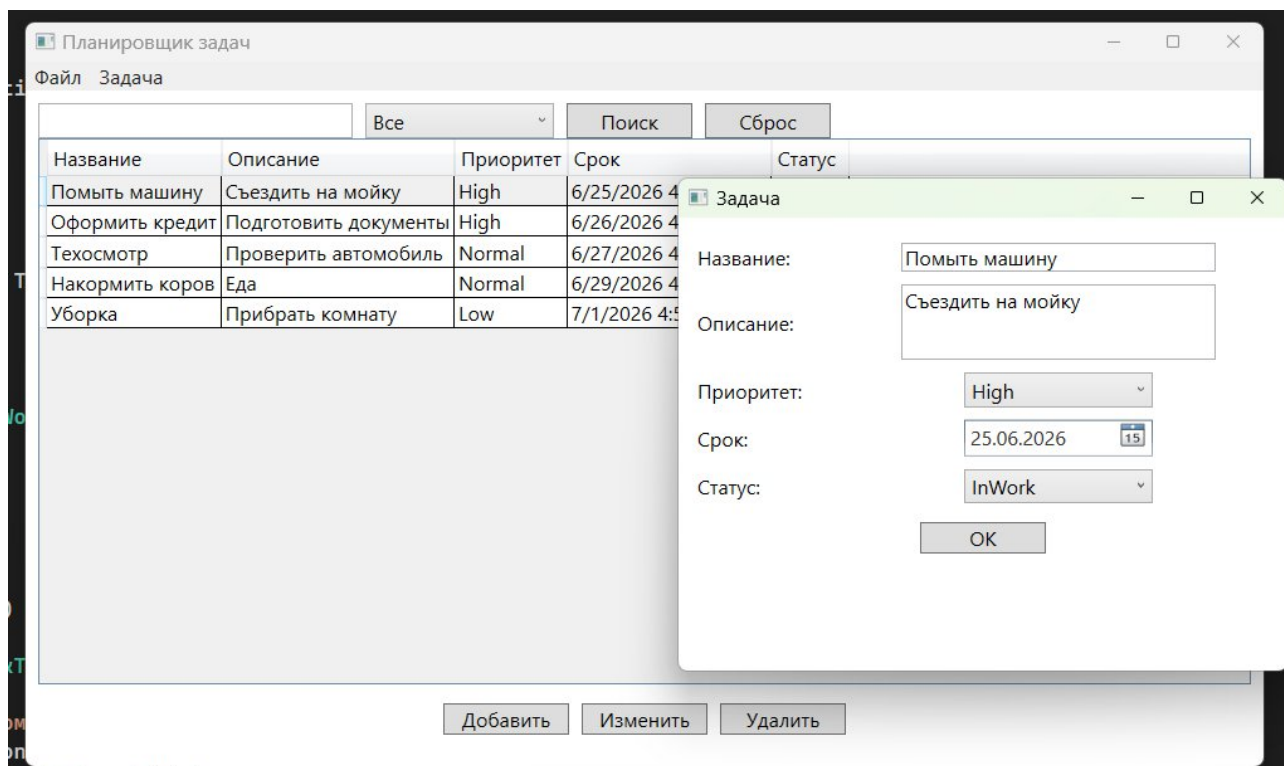


Рисунок 4 - окно редактирования задачи.

Этап 6. Разработка интерфейса. При проектировании интерфейса выполнены три стадии:

- проектирование - определены основные экраны (главное окно со списком задач, диалог добавления/редактирования задачи), составлен перечень элементов управления;
- прототипирование - на основе макета созданы XAML-разметки с размещением элементов: таблица DataGrid с колонками Название, Описание, Приоритет, Срок, Статус; панель поиска и фильтрации; панель кнопок;
- стилизация - применены отступы (Margin), выравнивание, ширины элементов для удобочитаемого и аккуратного внешнего вида.

РАЗДЕЛ 3. ОТЛАДКА ПРОГРАММНОГО МОДУЛЯ С ИСПОЛЬЗОВАНИЕМ СПЕЦИАЛИЗИРОВАННЫХ ПРОГРАММНЫХ СРЕДСТВ

3.1. Процесс отладки

Отладка программного модуля выполнялась с помощью встроенного отладчика Visual Studio 2022. В процессе разработки использовались следующие инструменты и приёмы:

- точки останова (Breakpoints) - устанавливались в методах `ServiceWorkTask.Add`, `Update`, `Delete` для пошагового контроля выполнения логики валидации и обновления коллекции;
- пошаговое выполнение (Step Over / Step Into) - применялось при проверке ветвления в методах фильтрации `GetStateTasks` и поиска `GetTextTasks`;
- окно Watch и окно Locals - использовались для наблюдения за состоянием объектов `WorkTask`, содержимым коллекции `_context.Tasks` и возвращаемыми значениями LINQ-запросов;
- обработка исключений - в WPF-слое все вызовы сервисных методов обернуты в блоки try-catch с выводом сообщения пользователю через `MessageBox`, что позволило выявить и устранить неочевидные ошибки ввода данных;
- немедленное окно (Immediate Window) - применялось для проверки выражений LINQ в режиме реального времени во время останова.

В ходе отладки были выявлены и устранены следующие проблемы:

- при попытке вызова `Edit_Click` без выбранной строки в `DataGrid` приложение выбрасывало `NullReferenceException` - добавлена проверка на null с выводом сообщения «Выберите задачу»;

- при загрузке JSON-файла с задачами не обновлялась таблица - устранено вызовом RefreshGrid() после ReplaceAll();
- ComboBox фильтра статуса не имел элемента «Все», что не позволяло сбросить фильтр - добавлен элемент «Все» при инициализации.

РАЗДЕЛ 4. РАЗРАБОТКА ТЕСТОВЫХ НАБОРОВ И ТЕСТОВЫХ СЦЕНАРИЕВ ДЛЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

4.1. Процесс тестирования. Unit-тесты для классов-сервисов

Для тестирования библиотеки TaskSchedulerData к решению добавлен проект MSTest - TaskSchedulerDataTests. В нём создана зависимость на TaskSchedulerData для тестирования её классов.

Разработаны три тестовых класса:

- ServiceWorkTaskTests - тестирование класса ServiceWorkTask (управление задачами)
- ServiceLoadDataJsonTests - тестирование класса ServiceLoadDataJson (работа с JSON)
- ServiceLoadDataXmlTests - тестирование класса ServiceLoadDataXml (работа с XML)

Тестирование ServiceWorkTask.

Инициализация: перед каждым тестом создаётся экземпляр ApplicationContext и новый экземпляр ServiceWorkTask. Контекст содержит тестовые данные, которые используются при выполнении части тестов.

```
[TestInitialize]
public void Initialize()
{
    _context = new ApplicationContext();
    _context.Tasks.Clear(); // очищаем тестовые данные
    _service = new ServiceWorkTask(_context);
}
```

Тест корректного добавления задачи (AddTest): создаётся задача с валидными данными, вызывается Add, проверяется что количество задач увеличилось после добавления новой записи.

```
[TestMethod]
public void AddTest()
{
    var task = new WorkTask("Проектирование")
    { DeadlineDate = DateTime.Now.AddDays(1) };
    _service.Add(task);
    Assert.AreEqual(1, _context.Tasks.Count);
}
```

Тест добавления null-задачи (Add_NullTask_ThrowsException): ожидается исключение ArgumentNullException при передаче null в метод Add.

Тест добавления задачи с пустым именем (Add_EmptyName_ThrowsException): ожидается ArgumentException при пустом TaskName.

Тест добавления задачи с неверной датой (Add_InvalidDate_ThrowsException): ожидается ArgumentException, если DeadlineDate в прошлом.

Тест корректного обновления (UpdateTest): задача добавляется в контекст напрямую, затем создаётся объект с тем же TaskId и новым именем, вызывается Update, проверяется новое имя.

```
[TestMethod]
public void UpdateTest()
{
    var task = new WorkTask("Первичный отчет")
    { DeadlineDate = DateTime.Now.AddDays(1) };
    _context.Tasks.Add(task);
    var updated = new WorkTask("Завершенный отчет")
```

```

        { TaskId = task.TaskId, DeadlineDate =
DateTime.Now.AddDays(2) };
        _service.Update(updated);
        Assert.AreEqual("Завершенный отчет",
_context.Tasks[0].TaskName);
    }

```

Тест обновления несуществующей задачи
(Update_NotExistingTask_ThrowsException): ожидается
InvalidOperationException.

Тест корректного удаления (DeleteTest): задача добавляется в контекст,
затем удаляется, проверяется что список пустой.

Тест удаления null (Delete_NullTask_ThrowsException): ожидается
ArgumentNullException.

Тест удаления несуществующей задачи
(Delete_NotExistingTask_ThrowsException): ожидается InvalidOperationException.

Тест фильтрации по статусу (GetStateTasksTest): добавляются две задачи
с разными статусами, GetStateTasks вызывается для одного из них, проверяется
что вернулась ровно одна задача.

Тест поиска по тексту (GetTextTasksTest): добавляется задача с заголовком
«Проект1» и описанием «Проектное решение», GetTextTasks вызывается с
запросом «проект», проверяется что вернулась одна задача
(регистронезависимый поиск).

Тестирование ServiceLoadDataJson.

Инициализация: создаётся экземпляр ServiceLoadDataJson и путь к
тестовому файлу test.json. Если файл существует - удаляется. Очистка
выполняется также после каждого теста.

Тест сохранения и загрузки (Save_And_Load_ReturnsTasks): создаётся
список из двух задач, сохраняется в файл через Save, затем загружается через
Load, проверяются количество, имена и статус задач.

[TestMethod]

```

public void Save_And_Load_ReturnsTasks()
{
    var tasks = new List<WorkTask>
    {
        new WorkTask("Проект") { DeadlineDate =
DateTime.Now.AddDays(1),
                                State = TaskState.Completed },
        new WorkTask("Отчет") { DeadlineDate =
DateTime.Now.AddDays(2) }
    };
    _service.Save(_path, tasks);
    var loaded = _service.Load(_path);
    Assert.AreEqual(2, loaded.Count);
    Assert.AreEqual(TaskState.Completed, loaded[0].State);
}

```

Тест загрузки несуществующего файла (Load_NotExistingFile_ThrowsException): ожидается FileNotFoundException при попытке загрузить «unknown.json».

Тестирование ServiceLoadDataXml.

Структура тестов аналогична JSON-сервису. Используется временный файл test.xml. Тест Save_And_Load_ReturnsTasks проверяет сохранение и восстановление задач в формате XML через XmlSerializer. Тест Load_NotExistingFile_ThrowsException проверяет обработку отсутствующего файла «unknown.xml».

4.2. Результаты тестирования

Все разработанные тесты успешно прошли проверку. Итоговые результаты запуска тестов в Test Explorer Visual Studio:

- ServiceWorkTaskTests: 10 тестов - все пройдены (Passed);
- ServiceLoadDataJsonTests: 2 теста - все пройдены (Passed);
- ServiceLoadDataXmlTests: 2 теста - все пройдены (Passed).

Все функциональные требования к классам-сервисам подтверждены модульными тестами.

Тестирование	Длительность	Признаки
✓ TasksSchedulerData...	105 мс	
✓ TasksSchedulerDat...	105 мс	
✓ ServiceLoadData...	45 мс	
✓ Load_NotExisti...	4 мс	
✓ Save_And_Loa...	41 мс	
✓ ServiceLoadData...	57 мс	
✓ Load_NotExisti...	4 мс	
✓ Save_And_Loa...	53 мс	
✓ ServiceWorkTask...	3 мс	
✓ Add_EmptyNa...	3 мс	
✓ Add_InvalidDa...	< 1 мс	
✓ Add_NullTask_...	< 1 мс	
✓ AddTest	< 1 мс	
✓ Delete_NotExi...	< 1 мс	
✓ Delete_NullTas...	< 1 мс	
✓ DeleteTest	< 1 мс	
✓ GetStateTasksT...	< 1 мс	
✓ GetTextTasksTe...	< 1 мс	
✓ ServiceWorkTa...	< 1 мс	
✓ Update_NotEx...	< 1 мс	
✓ UpdateTest	< 1 мс	

Рисунок 5 - результаты тестов.

РАЗДЕЛ 5. ИНСПЕКТИРОВАНИЕ КОМПОНЕНТ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ НА ПРЕДМЕТ СООТВЕТСТВИЯ СТАНДАРТАМ КОДИРОВАНИЯ

5.1. Процесс инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования

Инспектирование выполнялось методом ручного просмотра кода (Code Review) с использованием встроенных инструментов Visual Studio 2022.

Виды инспекции, применённые в проекте:

- статический анализ кода - выполнялся с помощью анализатора кода Visual Studio (Roslyn Analyzers), встроенного в IDE; предупреждения анализатора отображались на панели Error List
- ручной просмотр кода (Code Review) - каждый созданный класс и метод проверялся на соответствие соглашениям об именовании, структуре и документированию
- проверка покрытия тестами - осуществлялась через Test Explorer; основные методы сервисов проверены модульными тестами, что позволило подтвердить корректность их работы

Актуальные стандарты кодирования, используемые в проекте:

- Microsoft C# Coding Conventions (официальные соглашения Microsoft для языка C#) - определяют именование классов (PascalCase), методов (PascalCase), локальных переменных (camelCase), констант (UPPER_CASE), а также правила расположения using, структуру файла класса
- SOLID-принципы - применены принципы единственной ответственности (SRP) и инверсии зависимостей (DIP): бизнес-логика

вынесена в отдельную библиотеку, взаимодействие с ней осуществляется через интерфейсы IServiceWorkTask и IServiceLoadData

- соглашения по документированию - все публичные классы и интерфейсы снабжены XML-комментариями (///) там, где это необходимо

Результаты инспектирования по ключевым классам:

- WorkTask - именование свойств соответствует PascalCase; перечисления PriorityLevel и TaskState чётко описывают допустимые значения; конструкторы обеспечивают инициализацию по умолчанию
- ServiceWorkTask - методы следуют единой структуре (валидация - выполнение); валидация вынесена в приватный метод ValidateTask, устраняя дублирование кода; используются информативные сообщения исключений на русском языке
- ServiceLoadDataJson / ServiceLoadDataXml - оба класса реализуют общий интерфейс IServiceLoadData, обеспечивая единообразное использование и возможность замены реализации без изменения кода вызывающей стороны
- TaskGridControl - обработчики событий аккуратно разделены по функциям; вся работа с сервисами ведётся через приватные поля, инициализированные в конструкторе

В ходе инспектирования выявлены и устранены незначительные замечания:

- в ApplContext метод DataFill содержал тестовые данные (задачи «Помыть машину», «Накормить коров» и пр.) - для продуктовой версии рекомендуется либо убрать их, либо вынести в отдельный конфигурационный метод;

в TaskGridControl поля сервисов инициализируются прямо в конструкторе без использования DI-контейнера - для небольшого учебного проекта это допустимо.

ЗАКЛЮЧЕНИЕ

В результате прохождения учебной практики с 8 по 28 июня 2026 г. были закреплены теоретические знания и расширены профессиональные умения по специальности 09.02.07 «Информационные системы и программирование».

По итогам практики получены следующие результаты:

- разработано настольное WPF-приложение TaskScheduler на платформе .NET 8.0 для управления задачами;
- реализована библиотека классов TaskSchedulerData, содержащая модель данных, сервисы управления задачами и сервисы файловых операций в форматах JSON и XML;
- разработан удобный пользовательский интерфейс с возможностью добавления, редактирования, удаления, поиска и фильтрации задач;
- написаны 14 модульных тестов (MSTest), покрывающих все ключевые методы сервисов; все тесты успешно пройдены;
- проведено инспектирование кода на соответствие стандартам Microsoft C# Coding Conventions и принципам SOLID;
- выполнена отладка приложения с использованием встроенного отладчика Visual Studio 2022.

По окончании практики получен практический опыт разработки требований к программным модулям на основе анализа технической документации; интеграции модулей в программное обеспечение; отладки программного модуля с использованием специализированных программных средств; разработки тестовых наборов и тестовых сценариев; инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Документация Microsoft по C# и .NET 8.0. - URL: <https://docs.microsoft.com/ru-ru/dotnet/csharp/>
2. Документация Microsoft по WPF (Windows Presentation Foundation). - URL: <https://docs.microsoft.com/ru-ru/dotnet/desktop/wpf/>
3. Документация Microsoft по MSTest. - URL: <https://docs.microsoft.com/ru-ru/dotnet/core/testing/unit-testing-with-mstest>
4. Документация Microsoft по System.Text.Json. - URL: <https://docs.microsoft.com/ru-ru/dotnet/standard/serialization/system-text-json>
5. ГОСТ 19.201-78. Техническое задание. Требования к содержанию и оформлению. - М.: Стандартиформ, 2010.
6. ГОСТ 34.602-89. Техническое задание на создание автоматизированной системы. - М.: Стандартиформ, 2009.
7. Рихтер Дж. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5 на языке C#. - СПб.: Питер, 2013. - 896 с.
8. Нейгел К. и др. C# 7 и .NET Core 2.0. Современные кросс-платформенные разработки. - М.: Диалектика, 2018. - 1104 с.